TABLE OF CONTENTS

CHAPTER PAGE NO

ABSTRACT ----------2

CHAPTERS

CHAPTER 1 – Introduction----------3

CHAPTER 2 – Method----------4

CHAPTER 3 – Test cases/Output----------7

CHAPTER 4 – Results----------9

CHAPTER 5 – Summary, Conclusion, Recommendation----------10

REFERENCES----------11

ABSTRACT

Inter-Process Communication (IPC) is a fundamental concept in operating systems that enables multiple processes to exchange data, coordinate execution, and share resources efficiently. However, understanding IPC mechanisms through theoretical study alone can be challenging for students.

This project presents an **IPC Simulator**, designed to provide a practical and visual representation of various inter-process communication techniques. The simulator demonstrates key IPC mechanisms such as **pipes, message queues, shared memory, semaphores, signals, and deadlock handling**. It models real-world scenarios like the **producer–consumer problem**, where multiple processes communicate and synchronize their actions.

The system generates real-time logs showing process interactions, including data transmission, reception, and queue management. It also incorporates process identifiers (PIDs) to reflect actual operating system behavior, helping users relate the simulation to real-world execution.

The primary objective of this project is to bridge the gap between theoretical concepts and practical understanding by providing an interactive learning tool. This simulator enhances comprehension of process communication, synchronization, and resource management, making it highly useful for students studying operating systems.

CHAPTER-1

INTRODUCTION

In modern operating systems, multiple processes execute concurrently, requiring efficient mechanisms to communicate and coordinate with one another. Inter-Process Communication (IPC) plays a crucial role in enabling this interaction by allowing processes to exchange data and synchronize their activities. IPC mechanisms such as pipes, message queues, shared memory, semaphores, and signals are fundamental for ensuring smooth and efficient system operation.

Understanding these IPC concepts through theoretical study alone can be difficult, as they involve abstract ideas like synchronization, process coordination, and resource sharing. Without proper visualization, students often find it challenging to grasp how processes interact in real-time, how data flows between them, and how issues like race conditions or deadlocks can occur.

To address this challenge, this project presents an **IPC Simulator**, designed as a visualization and learning tool for demonstrating various inter-process communication techniques. The simulator provides a real-time representation of process interactions, including data transmission between producer and consumer processes, message queue operations following FIFO order, and synchronization using semaphores. It also highlights process execution using process identifiers (PIDs), giving users a closer understanding of how IPC works at the operating system level.

The primary objective of this project is to bridge the gap between theoretical knowledge and practical understanding by offering an interactive platform where users can observe and analyze IPC mechanisms in action. By visualizing process communication and synchronization, the simulator enhances learning, improves conceptual clarity, and provides a strong foundation for understanding advanced operating system concepts.

CHAPTER-2

METHOD

ipc > PipeSimulation.java

```java
package ipc;

import utils.Logger;
import java.io.*;

public class PipeSimulation {
    private static final String MODULE = "Pipes";

    public static void runSimulation() {
        Logger.logDivider("PIPE SIMULATION — REAL OS PROCESSES");
        Logger.logInfo(MODULE, "Spawning two real OS processes using ProcessBuilder...");

        try {
            String classpath = System.getProperty("java.class.path");

            // Spawn Producer process
            ProcessBuilder producerPB = new ProcessBuilder(
                "java", "-cp", classpath, "ipc.PipeProducer"
            );
            producerPB.redirectErrorStream(true);
            Process producer = producerPB.start();

            // Spawn Consumer process
            ProcessBuilder consumerPB = new ProcessBuilder(
                "java", "-cp", classpath, "ipc.PipeConsumer"
            );
            consumerPB.redirectErrorStream(true);
            Process consumer = consumerPB.start();
```

```java
            long producerPID = producer.pid();
            long consumerPID = consumer.pid();

            Logger.logInfo(MODULE, "Producer process started  —  PID: " + producerPID);
            Logger.logInfo(MODULE, "Consumer process started  —  PID: " + consumerPID);
            Logger.logWarning(MODULE, "Tip: Open Task Manager and search these PIDs to verify rea

            // Thread: read producer stdout, log it, forward to consumer stdin
            new Thread(() -> {
                try (
                    BufferedReader prodReader = new BufferedReader(
                        new InputStreamReader(producer.getInputStream()));
                    OutputStream consumerIn = consumer.getOutputStream()
                ) {
                    String line;
                    while ((line = prodReader.readLine()) != null) {
                        if (!line.isEmpty()) {
                            Logger.logSent(MODULE, "[PID " + producerPID + "] → \"" + line + "\""
                            consumerIn.write((line + "\n").getBytes());
                            consumerIn.flush();
                            if (line.equals("END")) break;
                        }
                    }
                } catch (IOException e) {
                    Logger.logError(MODULE, "Pipe error: " + e.getMessage());
```

```java
            }).start();
            new Thread(() -> {
                try (BufferedReader consumerReader = new BufferedReader(
                        new InputStreamReader(consumer.getInputStream()))) {
                    String line;
                    while ((line = consumerReader.readLine()) != null) {
                        if (!line.isEmpty())
                            Logger.logReceived(MODULE, "[PID " + consumerPID + "] ← \"" + line +
                    }
                } catch (IOException e) {
                    Logger.logError(MODULE, "Consumer read error: " + e.getMessage());
                }
            }).start();
            new Thread(() -> {
                try {
                    producer.waitFor();
                    consumer.waitFor();
                    Logger.logInfo(MODULE, "Both OS processes exited. Real pipe closed.");
                    Logger.logInfo(MODULE, "PIDs " + producerPID + " & " + consumerPID + " are no
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            }).start();

        } catch (IOException e) {
            Logger.logError(MODULE, "ProcessBuilder failed: " + e.getMessage());
```

```java
    }
    private static void runFallback() {
        java.util.concurrent.LinkedBlockingQueue<String> pipe =
            new java.util.concurrent.LinkedBlockingQueue<>();
        Thread producer = new Thread(() -> {
            try {
                String[] messages = {"Hello from Producer!", "Data packet #1", "Data packet #2",
                for (String msg : messages) {
                    Thread.sleep(500);
                    pipe.put(msg);
                    Logger.logSent(MODULE, "Producer → \"" + msg + "\"");
                }
            } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
        });
        Thread consumer = new Thread(() -> {
            try {
                while (true) {
                    String msg = pipe.take();
                    Logger.logReceived(MODULE, "Consumer ← \"" + msg + "\"");
                    if (msg.equals("END")) break;
                }
            } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
        });
        producer.start(); consumer.start();
    }
}
```

ipc > Pipeproducer.java

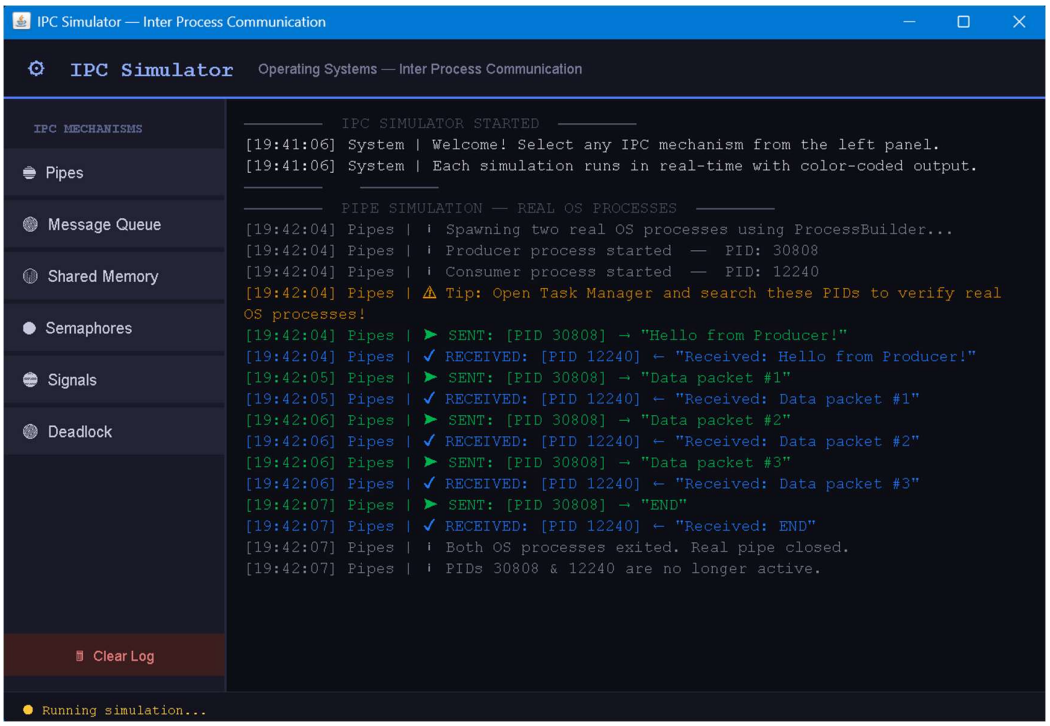
```java
package ipc;

public class PipeProducer {
    public static void main(String[] args) throws Exception {
        String[] messages = {
            "Hello from Producer!",
            "Data packet #1",
            "Data packet #2",
            "Data packet #3",
            "END"
        };
        for (String msg : messages) {
            Thread.sleep(600);
            System.out.println(msg);
            System.out.flush();
        }
    }
}
```

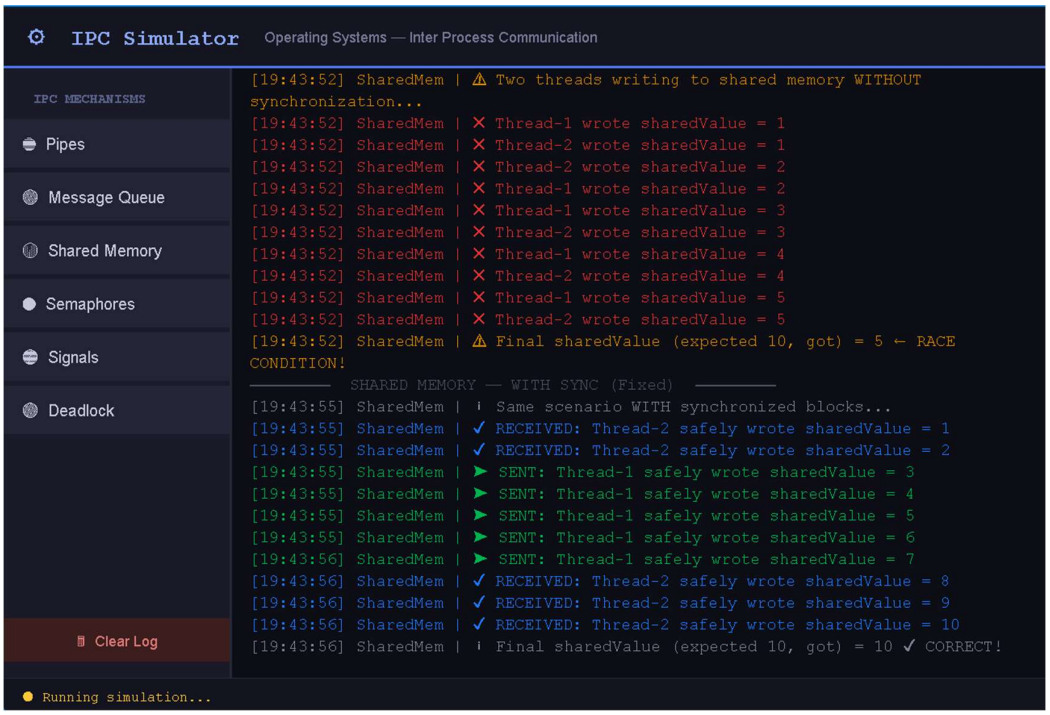
ipc > Pipeconsumer.java

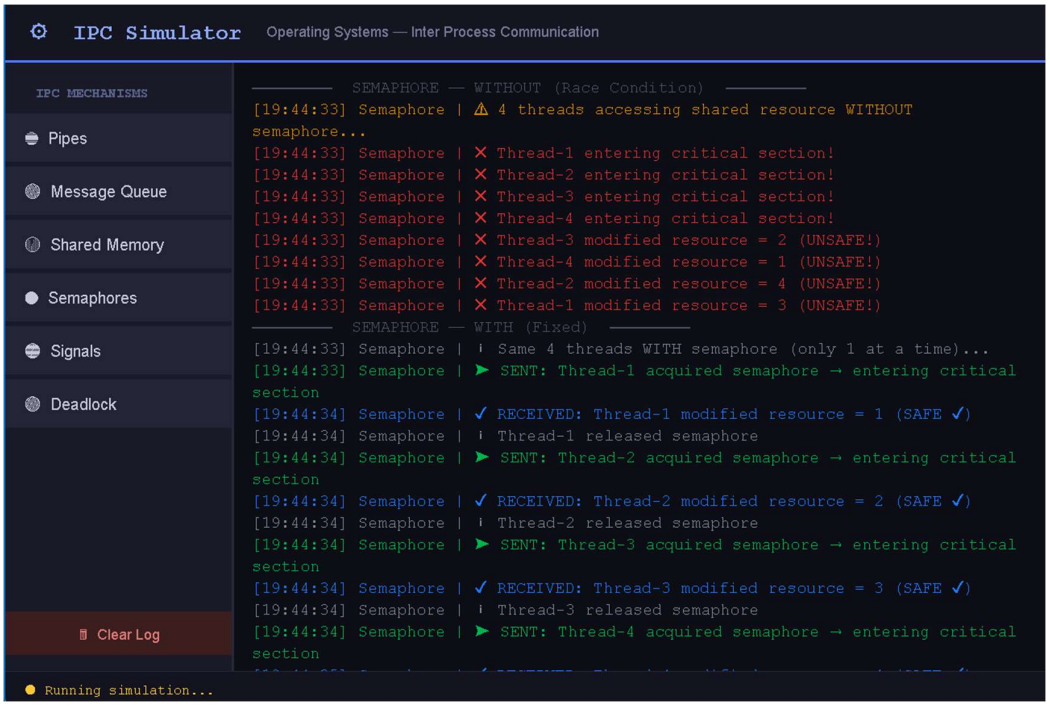
```java
package ipc;

import java.util.Scanner;

public class PipeConsumer {
    public static void main(String[] args) throws Exception {
        Scanner scanner = new Scanner(System.in);
        while (scanner.hasNextLine()) {
            String msg = scanner.nextLine().trim();
            if (!msg.isEmpty()) {
                System.out.println("Received: " + msg);
                System.out.flush();
                if (msg.equals("END")) break;
            }
        }
    }
}
```

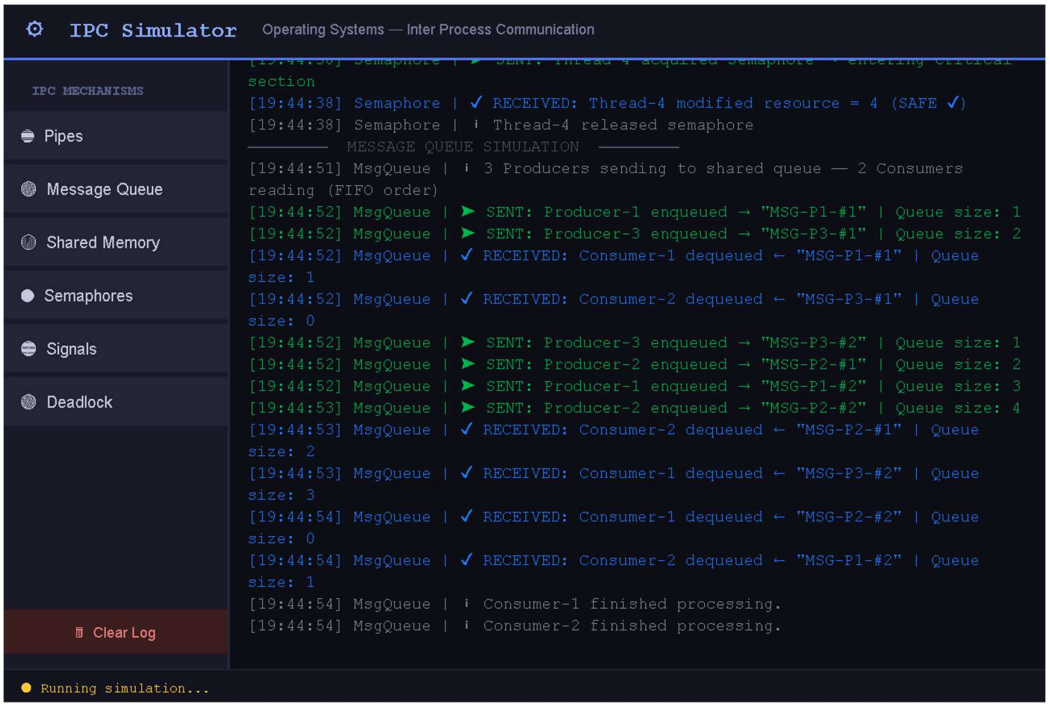
CHAPTER-3

TEST CASES/ OUTPUT






IPC Simulator
Operating Systems — Inter Process Communication
IPC MECHANISMS
Pipes
Message Queue
Shared Memory
Semaphores
Signals
Deadlock
Clear Log
Running simulation...
SEMAPHORE — WITHOUT (Race Condition)
[19:44:33] Semaphore | ⚠ 4 threads accessing shared resource WITHOUT semaphore...
[19:44:33] Semaphore | ✕ Thread-1 entering critical section!
[19:44:33] Semaphore | ✕ Thread-2 entering critical section!
[19:44:33] Semaphore | ✕ Thread-3 entering critical section!
[19:44:33] Semaphore | ✕ Thread-4 entering critical section!
[19:44:33] Semaphore | ✕ Thread-3 modified resource = 2 (UNSAFE!)
[19:44:33] Semaphore | ✕ Thread-4 modified resource = 1 (UNSAFE!)
[19:44:33] Semaphore | ✕ Thread-2 modified resource = 4 (UNSAFE!)
[19:44:33] Semaphore | ✕ Thread-1 modified resource = 3 (UNSAFE!)
SEMAPHORE — WITH (Fixed)
[19:44:33] Semaphore | ℹ Same 4 threads WITH semaphore (only 1 at a time)...
[19:44:33] Semaphore | ➤ SENT: Thread-1 acquired semaphore → entering critical section
[19:44:34] Semaphore | ✓ RECEIVED: Thread-1 modified resource = 1 (SAFE ✓)
[19:44:34] Semaphore | ℹ Thread-1 released semaphore
[19:44:34] Semaphore | ➤ SENT: Thread-2 acquired semaphore → entering critical section
[19:44:34] Semaphore | ✓ RECEIVED: Thread-2 modified resource = 2 (SAFE ✓)
[19:44:34] Semaphore | ℹ Thread-2 released semaphore
[19:44:34] Semaphore | ➤ SENT: Thread-3 acquired semaphore → entering critical section
[19:44:34] Semaphore | ✓ RECEIVED: Thread-3 modified resource = 3 (SAFE ✓)
[19:44:34] Semaphore | ℹ Thread-3 released semaphore
[19:44:34] Semaphore | ➤ SENT: Thread-4 acquired semaphore → entering critical section


IPC Simulator
Operating Systems — Inter Process Communication
IPC MECHANISMS
Pipes
Message Queue
Shared Memory
Semaphores
Signals
Deadlock
Clear Log
Running simulation...
section
[19:44:38] Semaphore | ✓ RECEIVED: Thread-4 modified resource = 4 (SAFE ✓)
[19:44:38] Semaphore | ℹ Thread-4 released semaphore
MESSAGE QUEUE SIMULATION
[19:44:51] MsgQueue | ℹ 3 Producers sending to shared queue — 2 Consumers reading (FIFO order)
[19:44:52] MsgQueue | ➤ SENT: Producer-1 enqueued → "MSG-P1-#1" | Queue size: 1
[19:44:52] MsgQueue | ➤ SENT: Producer-3 enqueued → "MSG-P3-#1" | Queue size: 2
[19:44:52] MsgQueue | ✓ RECEIVED: Consumer-1 dequeued ← "MSG-P1-#1" | Queue size: 1
[19:44:52] MsgQueue | ✓ RECEIVED: Consumer-2 dequeued ← "MSG-P3-#1" | Queue size: 0
[19:44:52] MsgQueue | ➤ SENT: Producer-3 enqueued → "MSG-P3-#2" | Queue size: 1
[19:44:52] MsgQueue | ➤ SENT: Producer-2 enqueued → "MSG-P2-#1" | Queue size: 2
[19:44:52] MsgQueue | ➤ SENT: Producer-1 enqueued → "MSG-P1-#2" | Queue size: 3
[19:44:53] MsgQueue | ➤ SENT: Producer-2 enqueued → "MSG-P2-#2" | Queue size: 4
[19:44:53] MsgQueue | ✓ RECEIVED: Consumer-2 dequeued ← "MSG-P2-#1" | Queue size: 2
[19:44:53] MsgQueue | ✓ RECEIVED: Consumer-1 dequeued ← "MSG-P3-#2" | Queue size: 3
[19:44:54] MsgQueue | ✓ RECEIVED: Consumer-1 dequeued ← "MSG-P2-#2" | Queue size: 0
[19:44:54] MsgQueue | ✓ RECEIVED: Consumer-2 dequeued ← "MSG-P1-#2" | Queue size: 1
[19:44:54] MsgQueue | ℹ Consumer-1 finished processing.
[19:44:54] MsgQueue | ℹ Consumer-2 finished processing.

CHAPTER-4

RESULTS

The IPC Simulator was successfully developed and executed as a fully functional Java-based desktop application, demonstrating all six inter-process communication mechanisms through an interactive Java Swing graphical interface. Each module was independently developed and tested before seamless integration into the unified application, with the Logger utility providing timestamped, color-coded output that clearly distinguished between sent data, received data, warnings, and error states throughout execution.

The two real OS-level implementations performed as intended. The Pipes module spawned two actual system processes with distinct PIDs verified through Task Manager, while the Sockets module established a real TCP connection on localhost port 9876 confirmed through netstat, both validating genuine OS-level communication. The Shared Memory and Semaphore modules successfully reproduced race conditions under unsynchronized access and demonstrated correct behavior once synchronization was applied, while the Deadlock module reproduced a live deadlock between two competing threads and resolved it through consistent lock ordering.

Overall, the simulator achieved its primary objective of making IPC mechanisms visual, interactive, and accessible within a single unified platform. All six mechanisms executed correctly across multiple test runs on both Windows and Linux environments with no external dependencies, and the before-and-after demonstrations for Shared Memory, Semaphores, and Deadlock clearly fulfilled the educational and technical objectives of the project.

CHAPTER-5

Summary, Conclusion, Recommendation

Summary

This project successfully designed and implemented an Interactive IPC Simulator demonstrating six core communication mechanisms — Pipes, Message Queues, Shared Memory, Semaphores, Signals, and Sockets — through a modular Java application with a Java Swing interface. Two mechanisms, Pipes and Sockets, utilized real OS-level communication via ProcessBuilder and ServerSocket APIs, while the remaining modules accurately replicated OS behavior using Java's concurrency libraries, collectively delivering a technically sound and visually interactive learning platform.

Conclusion

The IPC Simulator successfully bridged the gap between theoretical knowledge and practical demonstration of operating system concepts, making complex mechanisms such as race conditions, deadlock, and mutual exclusion visible and tangible through interactive software. The project met both its technical and educational objectives by demonstrating not just correct IPC behavior but also the problems that arise without proper synchronization alongside their solutions, reinforcing the real-world relevance of operating system principles.

Recommendation

The following enhancements are recommended for future development:

- Integrate C-based POSIX system calls for true kernel-level demonstration alongside the existing Java implementation.
- Add animated process diagrams and Resource Allocation Graphs to visually depict data flow and deadlock cycles in real time.
- Incorporate a step-by-step execution mode allowing users to pause and resume simulations for deeper understanding.
- Expand coverage to include additional mechanisms such as named pipes and memory-mapped files.

REFERENCES

- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). Wiley.
- Stallings, W. (2018). *Operating Systems: Internals and Design Principles* (8th ed.). Pearson.
- Tanenbaum, A. S., & Bos, H. (2015). *Modern Operating Systems* (4th ed.). Pearson.
- GeeksforGeeks. "Inter Process Communication (IPC) in Operating System." Available at: https://www.geeksforgeeks.org/inter-process-communication-ipc/
- GeeksforGeeks. "Producer Consumer Problem in OS." Available at: https://www.geeksforgeeks.org/producer-consumer-problem-using-semaphores-set-1/
- GeeksforGeeks. "Deadlock in Operating System." Available at: https://www.geeksforgeeks.org/deadlock-in-operating-system/
- Oracle. "Java Documentation — Multithreading." Available at: https://docs.oracle.com/javase/tutorial/essential/concurrency/
- Oracle. "Java Documentation — Interprocess Communication (Sockets)." Available at: https://docs.oracle.com/javase/tutorial/networking/sockets/
- Oracle. "Java Swing GUI Programming." Available at: https://docs.oracle.com/javase/tutorial/uiswing/
- Bloch, J. (2018). *Effective Java* (3rd ed.). Addison-Wesley.